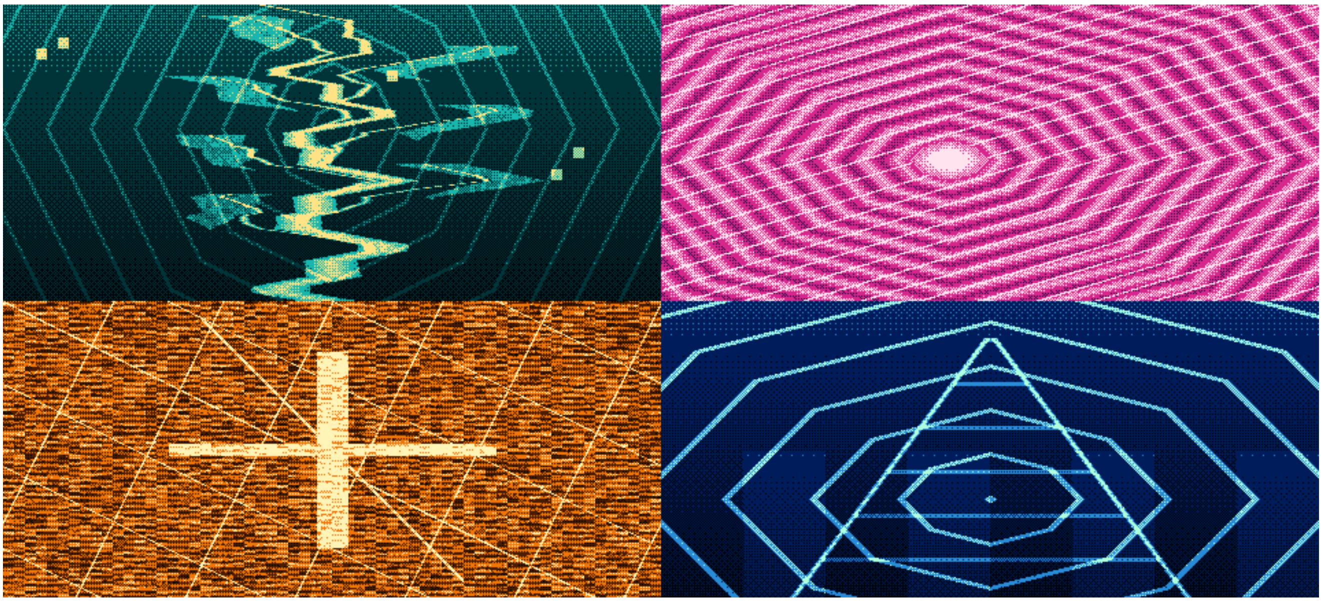
UGTS-KC 3.9.4

BAYER DIRECT NATIVE EDITION

Tiny arm64 Android application, direct integer fields, ordered dithering and explicit performance boundaries



Four deterministic 4-level field programs: Grove, shell, Kij lattice and SCLP cone/shell.

Release	Measured result
Native application	9,438-byte arm64-v8a APK, API 26+, test-signed
Native hot library	6,656 bytes; imports only libandroid.so and libc.so
Presentation path	ANativeWindow CPU buffer -> RGB565 -> Android compositor
Explicit exclusions	No DEX, asset, mesh, texture, shader, OpenGL ES, Vulkan, ray traversal or ray marching
Status	Executable candidate and source package; physical-phone install/performance not claimed

Prepared for Tom Klootwijk - Signature Edition - Version 3.9.4 - 28 August 2026

1. Executive decision

Version 3.9.4 makes compression and bounded presentation the first design constraints. It does not attempt to make the 3.9.3 IQ field renderer smaller. Instead, it defines a separate Bayer Direct profile whose installable hot path contains only native lifecycle code, four integer field programs, an exact 8x8 ordered-dither matrix and four RGB565 palettes.

DECISION - GO. Use a direct finite display lattice and Bayer palette projection. Exclude scene, mesh, texture, shader, graphics-pipeline and ray machinery from the native APK. Preserve the UGTS event/state substrate upstream and keep display output downstream.

Top-priority acceptance order

Priority	Release rule	Measured state
1. Size	Single ABI, no asset/DEX/shader payload, size-first native linking and compression.	APK 9,438 B; .so 6,656 B
2. Work bound	Finite 480-class RGB565 surface at nominal 30 Hz; no history buffer.	103,680 reference samples/frame; 202.5 KiB
3. Direct output	Evaluate F(x,y,q), apply Bayer threshold, post pixel. No scene traversal.	Four deterministic CRC baselines
4. Evidence	Separate host evidence from target-device claims.	11 tests pass; phone gates pending

Result in one sentence

The application is a tiny NativeActivity that synthesizes a retro, animated field image directly into the next Android native window buffer, with ordered dithering replacing continuous color and the operating system compositor performing only the final panel scaling.

2. Continuity with the formal substrate

This release is an additive presentation-profile delta over the 3.9.3 substrate. It does not renumber or remove earlier mechanisms. It appends M610-M629, bringing the combined engineering/source catalog range to M001-M629.

The decision follows explicit boundaries already present in the supplied project sources:

Source	Relevant boundary	3.9.4 use
UGTS-KC 3.6.2 SCLP, pp. 1-2, 11-12	The authoritative object is typed state/relations, not an image, mesh or ray-marched volume; rasterization and ray marching are explicitly excluded from the core.	Direct field queries feed an optional display projection.
UGTS-KC 2.0, pp. 2, 12-15	The core is not a renderer; reconstructibility and compact state require measured error and event density.	Frames reconstruct from seed/tick; no image history is authoritative.
KC Two Hands 3.0, pp. 5-10	Rendering stays downstream; hot records should remain compact while mesh/material data stays cold.	The cold graphics layer is entirely absent from the APK.
UGTS-GN 1.1, pp. 9-13	Packed records reduce traffic only under an error contract; compression is not correctness or speed by itself.	RGB565 and 480-class resolution are explicit, testable contracts.
KC Elizabeth 3.9, pp. 2-4	Small inspectable records, deterministic runtime and self-contained delivery are pragmatic design choices.	Single native library, no network assets, deterministic frame identity.

Important distinction: a screen still requires a finite pixel surface. 'No rasterization' in this profile means no triangle/mesh graphics rasterizer and no ray-based image solver. Each final pixel is a direct bounded query of an integer field. The Android compositor is an unavoidable downstream display endpoint, not substrate authority.

3. Bayer Direct architecture

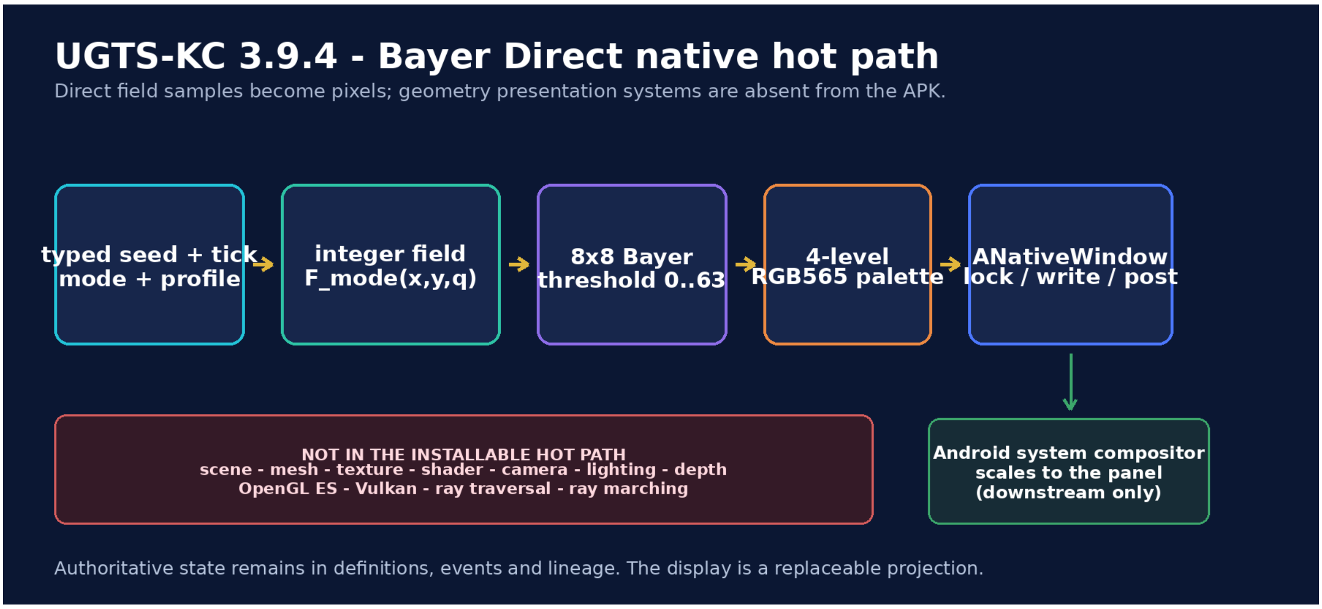


Figure 1. Direct field-to-pixel path. The only downstream system stage is native window posting and compositor scaling.

Formal state

```
q_BD = (seed, tick, mode, palette, width, height, flags)
```

For each finite output coordinate (x,y), the selected field produces L in [0,255]. The exact Bayer threshold selects one of four palette entries. The algorithm has no object list, visibility pass, depth buffer, material graph, camera or frame-history dependency.

Canonical UGTS handoff

```
field definition + state -> support / compatibility / guard (when application events require them)
-> deterministic transition + lineage -> optional Bayer Direct display projection
```

The display projection never commits gameplay, topology, ownership or lineage. It can be deleted or replaced without changing authoritative substrate records.

4. Bounded field programs

The APK includes four compact visual programs. They are deliberately formulaic rather than asset-driven. Each program is a finite nested loop over the current buffer and uses integer arithmetic, bitwise hashes, triangular waves and an approximate norm.

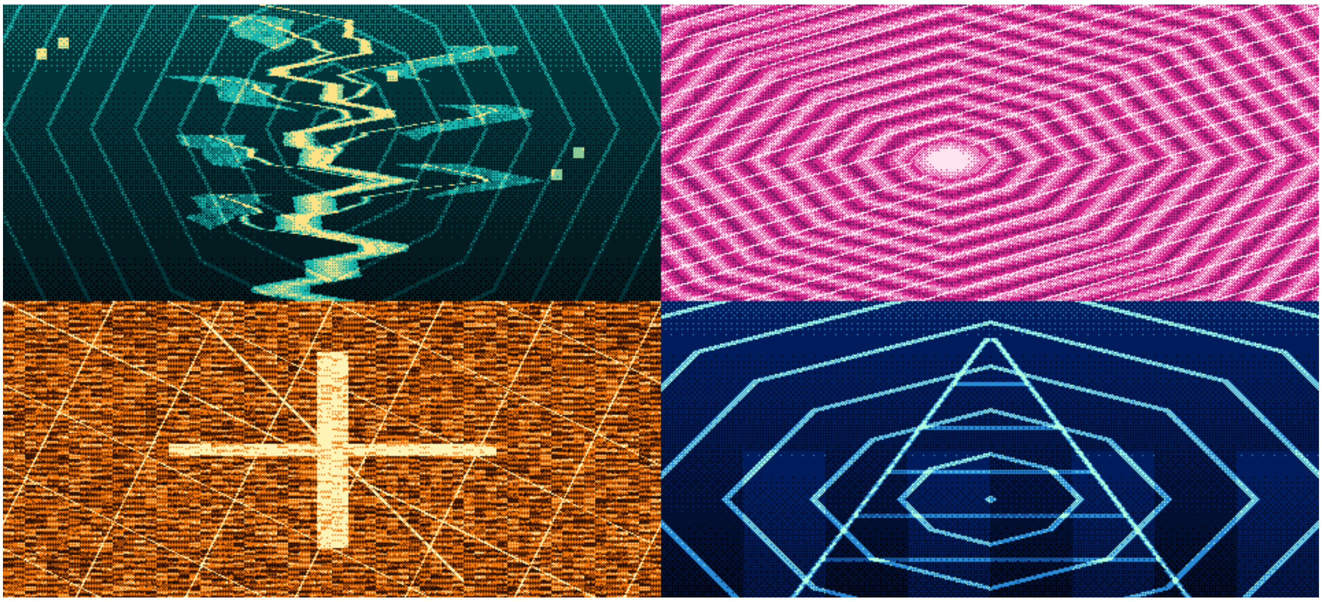


Figure 2. Reference output montage generated from the same C core used by the APK.

Mode	Field construction	Visual role
0 Grove	Vertical luminance, trunk/branch lines, bounded leaf halos, stars and ring phase.	Organic flagship identity without geometry assets.
1 Shell	Approximate radial norm, finite moving center, periodic shell phase and spokes.	High-motion ordered-dither test.
2 Kij lattice	Hashed 8-pixel cells, XOR lattice, diagonal guards and central mark.	Dense local-frequency and compression stress.
3 SCLP	Cone edge relation, finite stripe field, radial shell and parity wrap accent.	Direct visual reference to packed cone/shell mechanisms.

At nominal 30 Hz, the mode changes every 450 ticks, approximately every 15 seconds. No touch/input layer is bundled; that omission is intentional for this minimal release.

5. Ordered-dither quantization

Exact 8x8 Bayer threshold permutation

Every threshold 0..63 occurs once. The matrix is a deterministic quantizer, not frame histor

0	48	12	60	3	51	15	63
32	16	44	28	35	19	47	31
8	56	4	52	11	59	7	55
40	24	36	20	43	27	39	23
2	50	14	62	1	49	13	61
34	18	46	30	33	17	45	29
10	58	6	54	9	57	5	53

k = min(3, floor((4*L + 4*B8[x mod 8,y mod 8]) / 256))

Figure 3. Exact 64-state threshold matrix embedded as 64 bytes in the C source.

Let B8 be the threshold at the pixel's position modulo eight and let L be the field luminance. The output level is:

```
k = min(3, floor((4 * L + 4 * B8) / 256))
```

A four-entry palette then maps k to RGB565. Ordered dithering spreads quantization error deterministically in space. It does not create additional information, remove the need for a pixel buffer or guarantee perfect anti-aliasing. The exact matrix permutation and four fixed-mode CRC values are unit-tested.

6. Native Android display path

The application uses android.app.NativeActivity and exports ANativeActivity_onCreate from a single arm64 shared object. Window callbacks acquire and release ANativeWindow ownership, while a native thread locks, writes and posts the next finite buffer.

Window and lifecycle sequence

```
onCreate -> register native callbacks -> onNativeWindowCreated -> acquire -> setBuffersGeometry
-> start producer -> lock -> write -> unlockAndPost -> pause/focus idle -> stop/join before window
release
```

Contract	Implementation	Reason
Format	Request RGB565; support RGBA8888/RGBX8888 fallback.	16-bit hot buffer while tolerating device format decisions.
Resolution	480 along the long axis; derive other axis, align to 8, clamp 160..320.	Fixed sample budget independent of panel resolution.
Cadence	33,333 microsecond nominal wait; 50,000 microsecond idle wait.	Bounded work and low idle activity.
Dependencies	DT_NEEDED: libandroid.so and libc.so only.	No EGL, GLES, Vulkan, C++ runtime or app framework library.
Page alignment	AArch64 LOAD segments aligned to 0x4000.	Recorded ELF property; device loading still must be verified.

Physical-device status: the APK was not installed or benchmarked on the target phone in this build environment. Source, ELF, manifest and signatures are verified; Android loader/compositor behavior remains an acceptance gate.

7. Compression architecture

Compression is achieved mainly by deleting representation layers, not by applying a heavier archive codec. The field programs are code and small state; the image is reconstructed every frame. Nothing equivalent to a full scene, texture atlas or precomputed animation is stored.

Design choice	Size/performance consequence
Single arm64-v8a ABI	No duplicate native payload for x86, x86_64 or armeabi-v7a.
C hot path and hidden visibility	One exported symbol; section garbage collection removes unused code.
-Oz + LTO + stripped ELF	6,656-byte library while retaining lifecycle and four modes.
No C++ standard library	No libc++ payload or exception/unwind metadata.
No DEX and no app assets	No Java/Kotlin bytecode, textures, fonts, meshes, scene JSON or shaders.
RGB565 + low internal lattice	Two bytes/sample and bounded fill traffic; compositor handles final scale.
Seed/tick reconstruction	No frame cache or prior-image dependency; deterministic CRC regression.

Compression boundary

The 110.2x and 149.1x comparisons below are byte-width comparisons against the supplied 3.9.2 APK and stripped native library. They do not imply equal functionality: 3.9.4 intentionally removes the richer graphics, input, assets and scene path from the hot application.

8. Measured package size

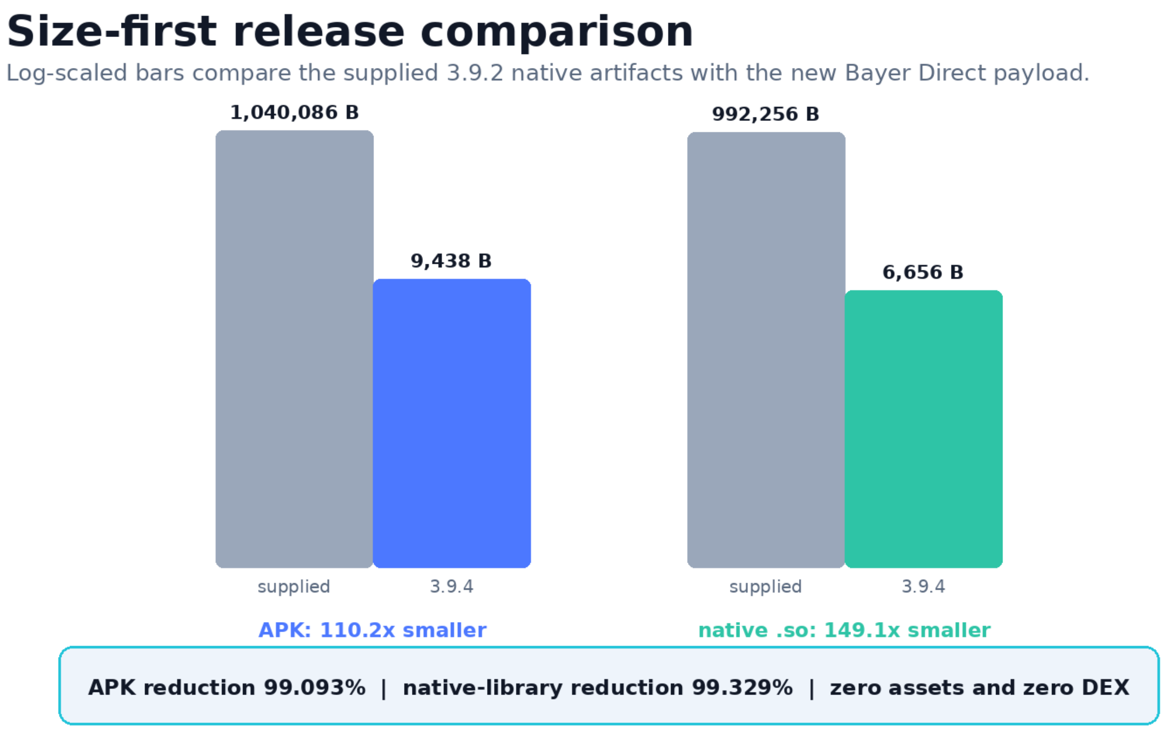


Figure 4. Exact file-size comparison. The chart uses logarithmic height because the new payload is two orders of magnitude smaller.

Artifact	Supplied 3.9.2	3.9.4	Reduction
Signed APK	1,040,086 B	9,438 B	110.20x smaller; 99.093%
Stripped native library	992,256 B	6,656 B	149.08x smaller; 99.329%

The final APK contains eight ZIP entries, two of which are directories. The only executable payload is the 6,656-byte arm64 shared object. The APK's compressed payload plus v1/v2 signing records totals 9,438 bytes.

```
APK SHA-256: eab84e326f52261bf34119c7d488bf4d4e6e1636cb24bfefa495d9250d4c09e3
Native library SHA-256: bbcff3d2ca84df6ee0ab839122aaf4c19787d094e343576657e65cb69c7f3482
```

9. Performance design and evidence

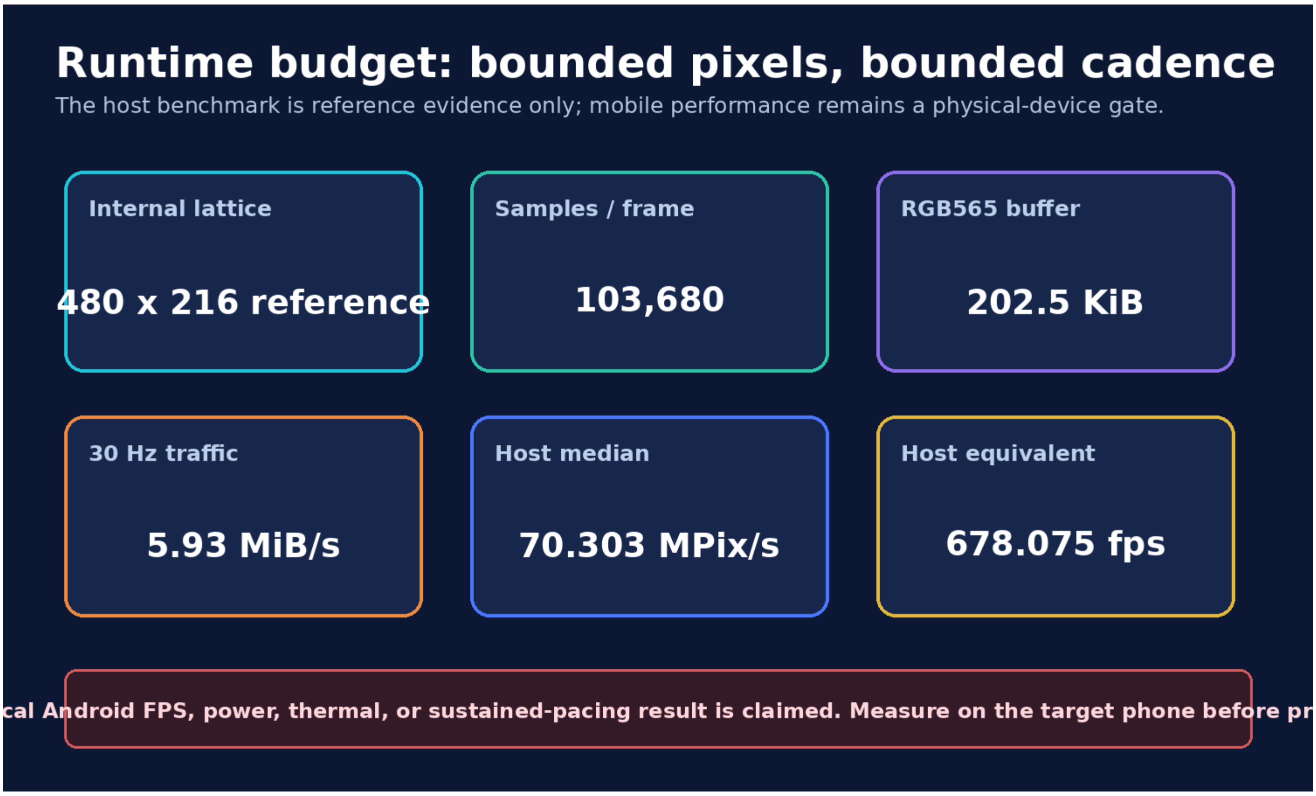


Figure 5. Bounded traffic and host-reference throughput. Host results are not target-phone results.

Reference host measurement

Five x86_64 Linux runs rendered 600 frames each at 480x216. Median throughput was 70.303 million pixels/s, equivalent to 678.075 full reference frames/s. Every run ended with CRC32 9f8347c1. This shows the C algorithm is deterministic and far below a desktop CPU's capacity; it does not predict Android sustained pacing, power or thermal behavior.

Target-device metrics required before a performance claim

Metric	Required evidence
Install/load	Package installs, NativeActivity launches, library resolves and survives lifecycle recreation.
Frame production	p50/p95/p99 producer time and missed 30 Hz intervals for 10- and 30-minute runs.
Buffer contract	Actual width, height, stride and format returned by ANativeWindow_lock.
Power/thermal	Battery discharge, CPU frequency residency, thermal status and device skin temperature.
Memory	RSS/PSS and compositor buffer allocation, not merely APK bytes.

10. APK contents, signature and install boundary

Final APK anatomy

Six payload records plus two directory entries; one compressed arm64 shared object carries the application.

AndroidManifest.xml	2,980 B source length / 1,080 B compressed
resources.arsc	864 B / 291 B compressed
libugts_kc_bayer.so	6,656 B / 3,679 B compressed
APK v1 metadata	MANIFEST.MF + SF + RSA
APK v2 block	1,562 B

Absent: classes.dex - assets - textures - meshes - shaders - Java/Kotlin runtime - GL/Vulkan libraries

Figure 6. Final install package anatomy.

The bundled candidate is signed with a self-signed 2048-bit RSA test certificate using both JAR/v1 records and an APK Signature Scheme v2 block. The package includes the public certificate fingerprint in validation evidence, but no private signing key is delivered.

Field	Value
Package ID	nl.tomklootwijk.ugtskc.bayer.poco
Version	versionCode 394; versionName 3.9.4-bayer-direct-v001
ABI / SDK	arm64-v8a only; minSdk 26; targetSdk 36
v2 certificate SHA-256	f100337d7c5fe902714c6d942fe48ce71300aa98aae4c27a6386bd0511f561f7
Certificate status	Self-signed test build; replace for production distribution.

Install candidate

```
adb install -r dist/UGTS_KC_Bayer_Direct_3_9_4_arm64-v8a.apk
```

The APK is structurally and cryptographically verified in the package, but an actual Android install was not available in this environment. Treat the first target-phone launch as an acceptance test, not as a foregone benchmark result.

11. Validation evidence

The release tests the deterministic quantizer, native binary, APK contents, binary manifest and both signature layers. These checks establish internal package consistency and the stated exclusions.

Gate	Result	Evidence
Bayer permutation	PASS	Values are exactly 0..63 once each.
Mode regression	PASS	Four fixed RGB565 CRC32 results.
Integer hot core	PASS	No float/double tokens or functions in core source.
ELF architecture/size	PASS	AArch64 DYN, 6,656 bytes.
Dynamic dependencies	PASS	Only libandroid.so and libc.so.
NativeActivity entry	PASS	ANativeActivity_onCreate exported.
APK size budget	PASS	9,438 bytes, below 16 KiB test ceiling.
APK contents	PASS	No assets, DEX or shader paths.
Manifest identity	PASS	Package/version/lib match; no uses-feature elements.
v1 signature	PASS	jarsigner verifies; self-signed warning expected.
v2 signature	PASS	Digest, certificate and signing block independently verified.

Combined result: 11/11 release tests pass. The wider 3.9.3 substrate test suite is not duplicated in this minimal app ZIP; this delta validates the presentation profile and packaged application.

12. Mechanism catalog M610-M619

These ten entries define the direct display, quantizer, state and field layer.

ID	Domain	Mechanism	Formal definition / status
M610	Profile	Bayer Direct presentation profile	A versioned downstream display profile maps deterministic integer field samples to a finite RGB565 surface without scene, mesh, texture, shader, ray, or camera state. [Implemented and tested]
M611	Display	Direct ANativeWindow buffer post	Lock the next native window buffer, write bounded samples, and unlock/post; no EGL, OpenGL ES, Vulkan, or Java canvas API is imported. [ELF import audit PASS]
M612	Packing	RGB565 hot buffer	Use 16 bits per sample with 5/6/5 color channels, halving byte traffic relative to RGBA8888 for the requested internal surface. [Runtime fallback to RGBA8888 included]
M613	Resolution	480-class internal lattice	Request width 480 in landscape and derive height from physical aspect, align to 8, and clamp to 160..320. Portrait applies the symmetric rule. [Implemented]
M614	Display	Compositor scale handoff	Let the Android compositor scale the finite internal buffer to the physical window; internal sample count remains independent of panel pixel count. [Requires device verification]
M615	Quantization	Exact Bayer 8x8 permutation	Use the standard 64-threshold ordered matrix containing every integer 0..63 exactly once. [Permutation unit test PASS]
M616	Quantization	Four-level palette projection	Compute k=min(3,floor((4L+4B8)/256)) and index one of four RGB565 palette entries. [Implemented and deterministic]
M617	Numeric	Integer-only field evaluator	The APK hot core uses 32-bit integer arithmetic, bounded division, bitwise hashes, triangular waves, and approximate norms; no float/double functions occur. [Source audit PASS]
M618	Persistence	Seed-tick reconstructible frame	A frame is reconstructed from seed, tick, mode, palette, width, height, and profile version; no prior image is authoritative. [CRC regression evidence]
M619	Field	Four bounded procedural programs	Grove, shell, Kij lattice, and SCLP cone/shell modes are finite closed loops over the output lattice and auto-cycle every 450 ticks. [Implemented]

13. Mechanism catalog M620-M629

These ten entries define exclusions, build compression, lifecycle, validation and governance.

ID	Domain	Mechanism	Formal definition / status
M620	Compression	No geometry materialization	The installable APK contains no scene pack, mesh, index buffer, material, texture, font, or artwork asset. [APK entry audit PASS]
M621	Query	No ray traversal or ray marching	No ray origin, ray direction, intersection loop, sphere-trace step, depth hierarchy, or acceleration structure is present in the app hot path. [Source and symbol scan PASS]
M622	Display	No graphics-API raster backend	No triangle pipeline or graphics API is linked. Pixel samples are direct field queries; the unavoidable display buffer and system compositor remain downstream. [ELF needs only libandroid and libc]
M623	ABI	Single arm64-v8a target	Ship only the 64-bit ARM ABI used by the requested modern Android target; other ABIs are source-build options, not bundled payload. [APK audit PASS]
M624	Build	Size-first native optimization	Compile with -Oz, LTO, hidden visibility, section GC, no build ID, no unwind tables, and stripped symbols. [6,656-byte shared object]
M625	Packaging	Zero asset and DEX payload	The APK carries only binary manifest, resource table, one arm64 shared object, and v1/v2 signature records. [9,438-byte APK]
M626	Timing	30 Hz paced production and idle wait	Post at a nominal 33,333 microsecond cadence; paused or unfocused state waits 50,000 microseconds and produces no frame. [Physical pacing measurement pending]
M627	Lifecycle	Native window ownership protocol	Acquire on window creation, stop/join producer before release on destruction, and restart after recreation. [Static lifecycle implementation]
M628	Validation	Deterministic frame CRC	CRC32 over RGB565 samples provides a regression checksum for fixed input state; it is not cryptographic identity. [Four mode CRC tests PASS]
M629	Governance	Compression and performance evidence boundary	Report exact package and host-reference metrics, but require named-device install, thermal, power, p50/p95/p99, and buffer-format evidence before promoting mobile performance claims. [Boundary enforced in report]

14. Reproduction, package map and evidence boundary

Standard source rebuild

Install JDK 17+, Android SDK platform 36, Android Gradle Plugin 8.13.2, CMake 3.22.1 and Android NDK r29 (29.0.14206865), then run:

```
gradle :app:assembleRelease
```

The included ultra-small APK was produced from the same C sources using a freestanding AArch64 link and a verified APK v2 signer because a complete Android SDK/NDK installation was unavailable in the build container. The source Gradle/CMake project is the recommended production rebuild path.

Package map

Path	Contents
app/	NativeActivity manifest, Gradle/CMake project and Bayer Direct C sources.
dist/	Test-signed arm64-v8a install candidate.
spec/	Formal definition, JSON profile and M610-M629 catalog.
preview/	Deterministic mode images and montage.
tests/	11 package/core regression tests.
tools/	Host preview, binary-manifest patcher, key generator and APK v2 sign/verify tools.
validation/	ELF/APK inspections, benchmark runs, signatures and release metrics.
report/	This PDF, report generator and figures.

Final evidence boundary

This release proves the exact contents and hashes of a very small native Android package, deterministic host output, a valid AArch64 ELF and internally verified APK signatures. It does not prove target-phone installation, compositor format selection, sustained frame pacing, thermal behavior, battery use, crash-free lifecycle or production signing. Those remain named-device measurements.

Attribution

Prepared as UGTS-KC 3.9.4 - Bayer Direct Native Edition for Tom Klootwijk. 'Signature Edition' is release attribution, not independent legal proof of authorship, ownership, patentability, priority or chain of title. Third-party Android platform names and standards remain the property of their respective owners.

Reference sources

Supplied project sources: UGTS-KC 3.6.2 SCLP; UGTS-KC 2.0; KC Two Hands 3.0; UGTS-GN 1.1; KC Elizabeth 3.9; and the immediate UGTS-KC 3.9.2/3.9.3 packages. Platform implementation references: Android NDK NativeActivity and ANativeWindow documentation.